Reproducibility Certification

Reproducibility Certification I

This section provides a machine-verifiable reproducibility certificate for the complete study. Every metric below is computed by the analysis pipeline and injected into the manuscript at render time — establishing a cryptographic chain of custody from configuration to publication.

Configuration Provenance I

Property	Value
Config hash (SHA-256, truncated)	7182b437c586a680
Paper version	2.5.2
First author	Daniel Ari Friedman
Keywords	optimization algorithms, gradient descent, convergence analysis, numerical methods, mathematical programming, reproducible research, infrastructure automation

Configuration Provenance II

The configuration hash changes whenever any parameter in `config.yaml` is modified, ensuring that every rendered PDF is traceable to a specific configuration state.

Generated Artifact Registry I

The analysis pipeline produced the following artifacts, each validated by `infrastructure.validation.output.validator`:

Category	Count
Publication-quality figures	8
Structured data files (CSV/JSON)	5
Analysis reports	9
Total artifacts	**22**

Numerical Validation Summary I

Convergence Verification

Within the configured grid, **4** of **6** runs satisfied `gradient_descent()` convergence (`No` indicates whether every row in `optimization_results.csv` converged).

- Converged step sizes: 0.1, 0.5, 1.0, 1.5
- Non-convergent or hit-iteration-cap step sizes: 0.01, 2.5
- Smallest recorded iteration count: 1 (at $\alpha = 1.0$)
- Largest recorded iteration count: 1000 (at $\alpha = 0.01$)
- Mean iterations across all rows: 372

Numerical Stability

Stability score from `infrastructure.scientific.stability`: **1.00** (out of 1.00)

The stability analysis tested 48 parameter combinations (8 starting points $\times$ 6 step sizes), confirming uniform convergence across the entire parameter space.

Numerical Validation Summary II

Benchmark Demonstration

This exemplar also demonstrates `infrastructure.benchmark`. The thin orchestrator `scripts/04_benchmark_stage.py` calls `src/benchmark_support.py`, which times the pure `quadratic_function` across a fixed set of input sizes (deterministic seed, no network) and turns the timing facts into boolean rubric checks. Those checks are scored through the shared `infrastructure.benchmark.score_rubric` against a weighted `RubricSet`, and the result is rendered with `scores_to_markdown` into `output/reports/benchmark_report.json` (plus an optional timing figure). This keeps the benchmark API exercised by a real exemplar, not by infrastructure tests alone.

Madlib Injection Verification I

This manuscript demonstrates the template's "madlib" capability: every quantitative claim is injected from computed data at render time. The substitution system processed the following variables:

- **Configuration variables**: Drawn from `manuscript/config.yaml` (`experiment:` section)
- **Result variables**: Computed from `output/data/optimization_results.csv`
- **Stability variables**: Extracted from `output/reports/stability_analysis.json`
- **Provenance variables**: Generated at substitution time (timestamps, hashes, versions)

To verify: modifying any value in `config.yaml` and re-running the pipeline will automatically update every corresponding claim in this document. No manual transcription is required or permitted.